T.C. ADNAN MENDERES ÜNİVERSİTESİ AYDIN MESLEK YÜKSEKOKULU DEĞİŞİMİN GELECEĞİ...	Programı : BİLGİSAYAR PROGRAMCILIĞI Dersin Adı : **Web Programcılığı III** Tarih :	
	Adı – Soyadı : **Berke HARMAN** Okul No : **256001044** İmza	
Öğretim Yılı-Dönemi: 2026-2027 GÜZ	**AraştırmaProjesi-1**	Not:

ARAŞTIRMA PROJESİ-1

Monolitik ve MikroservisMimarileri–AkıllıYurtAsistanı Proje Analizi

1) Araştırma Bölümü-1

a.MonolitikMimari ve Mikroservis Mimarisi

Monolitik mimari, bir uygulamanın bütün parçalarının (kullanıcı arayüzü, iş kuralları ve veritabanı erişimi) tek bir proje içinde yazıldığı, tek parça halinde derlenip tek seferde sunucuya yüklendiği yapıdır. Örneğin bir e-ticaret sitesinde ürün listeleme, sepet, ödeme ve kullanıcı işlemleri aynı kod tabanında bulunur, aynı süreç (process) içinde çalışır ve genellikle tek bir veritabanını ortak kullanır. Uygulamanın küçük bir yerinde değişiklik yapılsa bile bütün uygulama yeniden derlenip tekrar yayına alınır.

Bu yaklaşımın en büyük avantajı basitliğidir. Proje başında geliştirmesi, test etmesi ve sunucuya kurması kolaydır; modüller birbirini doğrudan fonksiyon çağrısıyla kullandığı için ağ gecikmesi yoktur. Ancak proje büyüdükçe kod tabanı karmaşıklaşır, bir modüldeki hata bütün sistemi durdurabilir ve farklı ekiplerin aynı kod üzerinde çalışması zorlaşır.

Mikroservis mimarisi ise uygulamanın, her biri tek bir iş alanından sorumlu küçük ve bağımsız servislere bölünmesidir. Aynı e-ticaret örneğinde "Ürün Servisi", "Sipariş Servisi", "Ödeme Servisi" ve "Kullanıcı Servisi" ayrı ayrı uygulamalar olarak çalışır. Her servisin genellikle kendi veritabanı vardır ve servisler birbiriyle HTTP/REST istekleri veya mesaj kuyrukları (RabbitMQ, Kafka gibi) üzerinden haberleşir. Her servis ayrı bir ekip tarafından geliştirilebilir, farklı bir programlama diliyle yazılabilir ve diğerlerini etkilemeden tek başına güncellenebilir.

Kısacası monolitik mimari "her şey tek bir büyük binada", mikroservis mimarisi ise "her birimin kendi küçük binası olduğu bir site" gibi düşünülebilir. Mikroservisler esneklik ve bağımsızlık kazandırırken servisler arası iletişim, izleme (monitoring) ve dağıtık veri yönetimi gibi ek zorlukları da beraberinde getirir.

b. Monolitik Mimaride Dikey ve Yatay Genişleme

Bir uygulamanın kullanıcı sayısı arttığında sistemin bu yükü kaldırabilmesi için genişletilmesi (ölçeklendirilmesi) gerekir. Monolitik bir uygulamada bu iki şekilde yapılabilir:

- **Dikey genişleme (Vertical Scaling / Scale-Up):** Uygulamanın çalıştığı mevcut sunucunun donanımının güçlendirilmesidir. İşlemci çekirdeği, RAM veya disk kapasitesi artırılır. Uygulama kodunda değişiklik gerektirmez ve uygulaması kolaydır. Fakat bir sunucunun alabileceği donanımın fiziksel bir sınırı vardır, güçlü donanımın maliyeti hızla artar ve tek sunucu çöktüğünde bütün sistem durur (tek hata noktası). Donanım yükseltme sırasında genellikle sistemin kısa süre kapatılması da gerekir.
- **Yatay genişleme (Horizontal Scaling / Scale-Out):** Uygulamanın birebir aynı kopyasının birden fazla sunucuda çalıştırılması ve gelen isteklerin bir yük dengeleyici (load balancer) ile bu sunucular arasında paylaştırılmasıdır. Yük arttıkça yeni sunucu eklenebilir, bir sunucu bozulsa bile diğerleri çalışmaya devam eder. Ancak monolitik uygulamada uygulamanın tamamı kopyalanmak zorundadır; yani sadece ödeme modülü yoğun olsa bile bütün uygulama çoğaltılır. Ayrıca oturum (session) bilgisinin ortak bir yerde (Redis vb.) tutulması ve tüm kopyaların aynı veritabanına bağlanması gerektiği için veritabanı zamanla darboğaz hâline gelebilir.

Aralarındaki farklar:

Kriter	Dikey Genişleme	Yatay Genişleme
Yöntem	Mevcut sunucuya daha fazla CPU/RAM eklenir.	Uygulamanın kopyaları yeni sunuculara kurulur.
Sınır	Donanımın fiziksel sınırı vardır.	Teorik olarak sunucu eklendikçe büyüyebilir.
Kesinti	Yükseltme sırasında genelde kesinti olur.	Sunucu eklerken sistem çalışmaya devam eder.
Hata toleransı	Tek sunucu çökerse sistem durur.	Bir sunucu çökse diğerleri hizmet verir.
Karmaşıklık	Basit, kod değişikliği gerekmez.	Yük dengeleyici, ortak oturum yönetimi gerekir.
Maliyet	Üst seviye donanım pahalıdır.	Ucuz sunucularla kademeli büyünebilir.

Süre : Ders saatinde çıktı alınarak teslim edilmelidir.	***Başarılar*** **Öğr.Gör. MehmetCan HANAYLI**

c. Mikroservis ve Monolitik Mimari ArasındakiFarklar

1. **Yapı ve kod tabanı:** Monolitik mimaride tüm uygulama tek bir kod tabanındadır. Mikroserviste her servis ayrı bir projedir ve kendi kod tabanına sahiptir.
2. **Dağıtım (deployment):** Monolitik uygulamada küçük bir değişiklik için bile tüm uygulama yeniden yayına alınır. Mikroserviste sadece değişen servis güncellenir, diğerleri çalışmaya devam eder.
3. **Ölçeklendirme:** Monolitik yapıda uygulamanın tamamı ölçeklenir. Mikroserviste yalnızca yoğunluk yaşanan servis (örneğin ödeme servisi) çoğaltılabilir; bu da kaynak tasarrufu sağlar.
4. **Hata yalıtımı:** Monolitik yapıda bir modüldeki bellek sızıntısı veya hata bütün uygulamayı çökertebilir. Mikroserviste bir servis çökerse diğer servisler çalışmaya devam edebilir.
5. **Veritabanı:** Monolitik mimaride genelde tek ve ortak bir veritabanı kullanılır. Mikroserviste her servisin kendi veritabanı olması önerilir; bu da veri tutarlılığını sağlamayı zorlaştırır.
6. **Teknoloji seçimi:** Monolitik uygulama tek bir dil ve çatıya (framework) bağlıdır. Mikroserviste her servis ihtiyacına göre farklı bir dil veya veritabanı kullanabilir.
7. **Geliştirme ve işletme zorluğu:** Monolitik mimari küçük projelerde daha hızlı geliştirilir, test edilir ve hata ayıklanır. Mikroservis mimarisi servisler arası iletişim, API Gateway, merkezi loglama, konteyner (Docker/Kubernetes) gibi ek altyapı ve tecrübe gerektirir.

Sonuç olarak küçük ve orta ölçekli, ekibi az olan projelerde monolitik mimari daha mantıklı bir başlangıçtır. Kullanıcı sayısı çok yüksek, birden fazla ekibin çalıştığı ve sık güncellenen büyük sistemlerde ise mikroservis mimarisi daha uygun olur.

2) Araştırma Bölümü-2

a. Proje: Akıllı Yurt Asistanı

Seçtiğim proje, **Akıllı Yurt Asistanı** adlı, IoT (Nesnelerin İnterneti) destekli bir yurt yönetim sistemidir. Sistem; öğrencilerin ve yurt personelinin kullanacağı bir mobil uygulama, iş kurallarını yürüten bir Web API (backend), ilişkisel bir veritabanı ve odalara yerleştirilecek ESP32 tabanlı sensör/kart okuyucu cihazlardan oluşmaktadır. Arıza bildirimi, duyurular, yemek menüsü, çamaşırhane rezervasyonu, oda ortam takibi (sıcaklık, nem, gaz) ve RFID kart ile giriş-çıkış takibi gibi birçok modül içerdiği için detaylı analiz gerektiren bir projedir.

Kullanılacak teknolojiler: C#, ASP.NET Core Web API, .NET MAUI (mobil), Microsoft SQL Server, Entity Framework Core, ESP32 (C++), DHT22, MQ-2 ve RC522 RFID modülleri.

b. Projenin Amacı

Projenin amacı, yurtlarda hâlâ büyük ölçüde kâğıt, defter ve sözlü iletişimle yürütülen günlük işleri tek bir dijital platformda toplamaktır. Öğrencinin arıza bildirmek için idareye gitmesi, çamaşırhanede boş makine beklemesi veya duyuruları panoda araması yerine bu işleri telefonundan birkaç dokunuşla yapabilmesi hedeflenmektedir. Bunun yanında odalara yerleştirilen sensörlerle sıcaklık, nem ve gaz seviyesinin sürekli izlenmesi, tehlikeli bir durumda yönetimin anında uyarılması ve yurt giriş-çıkışlarının kart okutularak otomatik kaydedilmesi ile daha **güvenli, konforlu ve takip edilebilir** bir yurt ortamı oluşturulması amaçlanmaktadır.

c. Projenin Kapsamı

Proje kapsamına dahil olanlar:

- Kullanıcı kayıt/giriş ve rol yönetimi (öğrenci, yurt yöneticisi, teknik personel)
- Bina, oda ve öğrenci–oda eşleştirme yönetimi
- Arıza/talep oluşturma, personele atama ve durum takibi
- Duyuru yayınlama ve uygulama içi bildirimler
- Günlük/haftalık yemek menüsü görüntüleme ve yemek puanlama

- Çamaşır makinesi doluluk görüntüleme ve saatlik rezervasyon
- ESP32 ile oda sıcaklık, nem ve gaz ölçümü, eşik aşımında alarm
- RFID kart okuyucu ile yurt giriş-çıkış kaydı
- Yönetici için raporlama ve analiz ekranları (arıza istatistikleri, ortam grafikleri, giriş-çıkış raporları)

Proje kapsamı dışında kalanlar:

- Yurt ücreti tahsilatı ve online ödeme işlemleri
- Kamera/görüntü işleme ve yüz tanıma ile giriş
- Yurt kayıt başvurusu ve öğrenci seçim/yerleştirme süreci
- Klima, kombi gibi cihazların uzaktan kontrolü (sadece izleme yapılacaktır)

d. Projenin Çözdüğü Problem

Yurtlarda yaşanan başlıca sorunlar şunlardır:

- **Arıza takibinin zorluğu:** Arızalar sözlü veya kâğıt formla bildiriliyor, form kaybolabiliyor ve öğrenci talebinin hangi aşamada olduğunu bilemiyor. Yönetim de hangi arızanın ne kadar sürede çözüldüğünü ölçemiyor.
- **Bilgiye ulaşamama:** Duyurular ve yemek menüsü yalnızca panoya asılıyor, öğrenciler önemli bilgileri kaçırıyor.
- **Çamaşırhane yoğunluğu:** Öğrenciler boş makine olup olmadığını bilmeden çamaşırhaneye iniyor ve uzun süre bekliyor, makineler adil kullanılmıyor.
- **Güvenlik riskleri:** Odalarda gaz kaçağı veya aşırı ısınma gibi durumlar ancak fark edildiğinde müdahale edilebiliyor.
- **Giriş-çıkışların elle tutulması:** Giriş-çıkış defteri zaman alıyor, hatalı veya eksik tutulabiliyor; geriye dönük sorgulama yapmak çok zor.

Akıllı Yurt Asistanı bu sorunları; dijital talep takibi, anlık bildirimler, rezervasyon sistemi, sensörle sürekli izleme ve otomatik kart kaydı ile çözmektedir.

e. Projenin Hedef Kitlesi

- **Yurtta kalan öğrenciler:** Uygulamanın asıl kullanıcılarıdır. Arıza bildirir, duyuru ve menüyü takip eder, çamaşırhane rezervasyonu yapar, oda ortam bilgilerini görür.
- **Yurt yöneticileri ve idari personel:** Talepleri yönetir, duyuru yayınlar, odaları ve alarmları izler, raporları inceler.
- **Teknik personel:** Kendisine atanan arıza işlerini görür ve tamamladığında durumunu günceller.
- **Kurum yönetimi (üniversite, KYK veya özel yurt işletmeleri):** Raporlar sayesinde yurtların genel durumu hakkında karar alır.
- **Dolaylı olarak öğrenci velileri:** Yurdun daha güvenli ve takip edilebilir olmasından fayda sağlar.

f. Proje Zaman Planı

Proje 12 hafta olarak planlanmıştır. Aşamalar ve haftalara dağılımı aşağıda gösterilmiştir.

No	İş / Aşama	Çıktı	Süre	H 1	H 2	H 3	H 4	H 5	H 6	H 7	H 8	H 9	H 10	H 11	H 12
1	Fizibilite ve gereksinim analizi	Proje önerisi, gereksinim listesi	1 hf	■											
2	Sistem tasarımı (ER, UML, Use Case, arayüz taslakları)	Tasarım dokümanı	1 hf		■										
3	Veritabanı oluşturma	SQL Server veritabanı	1 hf			■									
4	Backend – kimlik doğrulama ve kullanıcı/rol	Çalışan giriş API'si	2 hf			■	■								
5	Backend – modül API'leri (arıza, duyuru, menü, çamaşırhane)	REST API uçları	3 hf				■	■	■						

Süre : Ders saatinde çıktı alınarak teslim edilmelidir.	***Başarılar*** **Öğr.Gör. MehmetCan HANAYLI**

T.C. ADNAN MENDERES ÜNİVERSİTESİ AYDIN MESLEK YÜKSEKOKULU DEĞİŞİMİN GELECEĞİ...	Programı : BİLGİSAYAR PROGRAMCILIĞI Dersin Adı : **Web Programcılığı III** Tarih :	
	Adı – Soyadı : **Berke HARMAN** Okul No : **256001044** İmza	
Öğretim Yılı-Dönemi: 2026-2027 GÜZ	**AraştırmaProjesi-1**	Not:

No	İş / Aşama	Çıktı	Süre		H 2	H 3	H 4	H 5	H 6	H 7	H 8	H 9	H 10	H 11	H 12
6	ESP32 donanım kurulumu ve yazılımı	Sensör/RFID prototipi	3 hf												
7	Mobil uygulama ekranları	.NET MAUI uygulaması	5 hf												
8	Raporlama ekranları ve bildirim sistemi	Rapor ve alarm ekranları	2 hf												
9	Entegrasyon ve test (birim, sistem, kullanıcı testi)	Test raporu	2 hf												
10	Dokümantasyon, kurulum ve teslim	Kullanıcı kılavuzu, teslim	1 hf												

H: Hafta. Koyu hücreler ilgili işin yapılacağı haftaları göstermektedir. Bazı işler (örneğin ESP32 ve mobil uygulama) paralel yürütülecektir.

g. Gereksinimler ve MoSCoW Sınıflandırması

MoSCoW yöntemi, gereksinimleri önem derecesine göre dört gruba ayıran bir önceliklendirme tekniğidir: **Must have** (mutlaka olmalı – olmazsa proje başarısız sayılır), **Should have** (olmalı – önemli ama ilk sürümde zorunlu değil), **Could have** (olabilir – zaman kalırsa eklenecek güzel özellikler) ve **Won't have** (bu sürümde olmayacak – ileriki sürümlere bırakılan özellikler). Projem için belirlediğim 28 gereksinim ve öncelikleri aşağıdadır.

No	Gereksinim	MoSCoW
G01	Yurt yönetimi arıza taleplerini kâğıt form yerine tamamen dijital ortamda toplayabilmelidir.	Must
G02	Arıza taleplerinin ortalama çözülme süresi ilk dönem sonunda %30 kısaltılmalıdır.	Should
G03	Yurt giriş-çıkış kayıtları elle tutulan defter yerine otomatik olarak tutulmalıdır.	Must
G04	Kurum yönetimi aylık arıza, doluluk ve ortam raporlarını tek tıkla alabilmelidir.	Should
G05	Öğrenci, odasındaki arızayı telefonundan en fazla 3 adımda bildirebilmek ister.	Must
G06	Öğrenci, çamaşırhaneye inmeden hangi makinenin boş olduğunu görebilmek ister.	Should
G07	Öğrenci, günün yemek menüsünü görüp yemeği puanlayabilmek ister.	Could
G08	Yurt yöneticisi gelen tüm talepleri tek ekranda görüp personele dağıtabilmek ister.	Must
G09	Teknik personel kendisine atanan işleri listeleyip "tamamlandı" olarak işaretleyebilmek ister.	Must
G10	Sistem kullanıcıların e-posta ve şifre ile giriş yapmasını sağlamalı, şifreleri hash'leyerek saklamalıdır.	Must
G11	Sistem, kullanıcının rolüne (öğrenci/yönetici/personel) göre menüleri ve yetkileri değiştirmelidir.	Must
G12	Sistem arıza talebine kategori, açıklama ve isteğe bağlı fotoğraf eklenmesine izin vermelidir.	Must
G13	Sistem talebin durumu değiştiğinde ilgili öğrenciye bildirim göndermelidir.	Should
G14	Sistem ESP32'den gelen sıcaklık, nem ve gaz değerlerini kaydedip oda bazında göstermelidir.	Must
G15	Sistem gaz veya sıcaklık eşik değeri aşıldığında yöneticiye ve oda sakinlerine alarm bildirimi göndermelidir.	Must
G16	Sistem RFID kart okutulduğunda kişi, yön (giriş/çıkış) ve zaman bilgisiyle kayıt oluşturmalıdır.	Must
G17	Sistem çamaşır makineleri için saatlik rezervasyon yapılmasını sağlamalı, çakışan rezervasyonu	Should
G18	engellemelidir.	Should
G19	Sistem yöneticinin duyuru yayınlamasını ve duyuruların tüm öğrencilere bildirilmesini sağlamalıdır.	Could
G20	Sistem raporları PDF veya Excel olarak dışa aktarabilmelidir.	Could
G21	Sistem yöneticinin eşik değerlerini (ör. sıcaklık > 30 °C) oda bazında ayarlamasına izin vermelidir.	Won't
G22	Yurt odalarının 3 boyutlu kat planı üzerinde görselleştirilmesi (bu sürümde yapılmayacak).	Must
G23	Tüm istemci-sunucu iletişimi HTTPS ile şifrelenmeli, oturumlar JWT ile yönetilip 24 saat sonra sona	Should
G24	ermelidir.	Should
G25	Uygulama ekranları en geç 2 saniyede açılmalı, API ortalama yanıt süresi 500 ms'nin altında olmalıdır.	Must
G26	Sistem en az 500 eşzamanlı kullanıcıya kesintisiz hizmet verebilmelidir.	Should
G27	Uygulama Android 8.0 ve üzeri cihazlarda çalışmalı, arayüzü Türkçe ve kolay kullanılır olmalıdır.	Could
G28	Veritabanı her gece otomatik olarak yedeklenmeli, kişisel veriler KVKK'ya uygun saklanmalıdır.	Won't
	Sistemin %99 oranında erişilebilir (çalışır) durumda olması.	
	Yüz tanıma ile yurt girişi (bu sürümde yapılmayacak).	

Özet: Must: 13 · Should: 9 · Could: 4 · Won't: 2 (toplam 28 gereksinim)

Süre : Ders saatinde çıktı alınarak teslim edilmelidir.	***Başarılar*** **Öğr.Gör. MehmetCan HANAYLI**

h. Gereksinimlerin Türlerine Göre Sınıflandırılması

i. İş Gereksinimleri: Kurumun (yurt yönetiminin) bu projeden elde etmek istediği üst düzey hedeflerdir. "Neden bu sistemi yapıyoruz?" sorusunun cevabıdır.

No	Gereksinim	MoSCoW
G01	Yurt yönetimi arıza taleplerini kâğıt form yerine tamamen dijital ortamda toplayabilmelidir.	Must
G02	Arıza taleplerinin ortalama çözülme süresi ilk dönem sonunda %30 kısaltılmalıdır.	Should
G03	Yurt giriş-çıkış kayıtları elle tutulan defter yerine otomatik olarak tutulmalıdır.	Must
G04	Kurum yönetimi aylık arıza, doluluk ve ortam raporlarını tek tıkla alabilmelidir.	Should

ii. Kullanıcı Odaklı Gereksinimler: Kullanıcıların sistemi kullanarak yapabilmek istedikleri işlerdir; kullanıcının bakış açısından yazılır.

No	Gereksinim	MoSCoW
G05	Öğrenci, odasındaki arızayı telefonundan en fazla 3 adımda bildirebilmek ister.	Must
G06	Öğrenci, çamaşırhaneye inmeden hangi makinenin boş olduğunu görebilmek ister.	Should
G07	Öğrenci, günün yemek menüsünü görüp yemeği puanlayabilmek ister.	Could
G08	Yurt yöneticisi gelen tüm talepleri tek ekranda görüp personele dağıtabilmek ister.	Must
G09	Teknik personel kendisine atanan işleri listeleyip "tamamlandı" olarak işaretleyebilmek ister.	Must

iii. Fonksiyonel Gereksinimler: Sistemin ne yapması gerektiğini, yani sunacağı işlevleri ve davranışları tanımlar.

No	Gereksinim	MoSCoW
G10	Sistem kullanıcıların e-posta ve şifre ile giriş yapmasını sağlamalı, şifreleri hash'leyerek saklamalıdır.	Must
G11	Sistem, kullanıcının rolüne (öğrenci/yönetici/personel) göre menüleri ve yetkileri değiştirmelidir.	Must
G12	Sistem arıza talebine kategori, açıklama ve isteğe bağlı fotoğraf eklenmesine izin vermelidir.	Must
G13	Sistem talebin durumu değiştiğinde ilgili öğrenciye bildirim göndermelidir.	Should
G14	Sistem ESP32'den gelen sıcaklık, nem ve gaz değerlerini kaydedip oda bazında göstermelidir.	Must
G15	Sistem gaz veya sıcaklık eşik değeri aşıldığında yöneticiye ve oda sakinlerine alarm bildirimi göndermelidir.	Must
G16	Sistem RFID kart okutulduğunda kişi, yön (giriş/çıkış) ve zaman bilgisiyle kayıt oluşturmalıdır.	Must
G17	Sistem çamaşır makineleri için saatlik rezervasyon yapılmasını sağlamalı, çakışan rezervasyonu engellemelidir.	Should
G18	Sistem yöneticinin duyuru yayınlamasını ve duyuruların tüm öğrencilere bildirilmesini sağlamalıdır.	Should
G19	Sistem raporları PDF veya Excel olarak dışa aktarabilmelidir.	Could
G20	Sistem yöneticinin eşik değerlerini (ör. sıcaklık > 30 °C) oda bazında ayarlamasına izin vermelidir.	Could
G21	Yurt odalarının 3 boyutlu kat planı üzerinde görselleştirilmesi (bu sürümde yapılmayacak).	Won't

iv. Fonksiyonel Olmayan Gereksinimler: Sistemin nasıl çalışması gerektiğini; performans, güvenlik, kullanılabilirlik, uyumluluk gibi kalite özelliklerini tanımlar.

No	Gereksinim	MoSCoW
G22	Tüm istemci-sunucu iletişimi HTTPS ile şifrelenmeli, oturumlar JWT ile yönetilip 24 saat sonra sona ermelidir.	Must
G23	Uygulama ekranları en geç 2 saniyede açılmalı, API ortalama yanıt süresi 500 ms'nin altında olmalıdır.	Should
G24	Sistem en az 500 eşzamanlı kullanıcıya kesintisiz hizmet verebilmelidir.	Should
G25	Uygulama Android 8.0 ve üzeri cihazlarda çalışmalı, arayüzü Türkçe ve kolay kullanılır olmalıdır.	Must
G26	Veritabanı her gece otomatik olarak yedeklenmeli, kişisel veriler KVKK'ya uygun saklanmalıdır.	Should
G27	Sistemin %99 oranında erişilebilir (çalışır) durumda olması.	Could
G28	Yüz tanıma ile yurt girişi (bu sürümde yapılmayacak).	Won't

i. Kullanım Senaryoları (Use Case)

Fonksiyonel gereksinimlerden G12 (arıza talebi oluşturma), G15 (eşik aşımında alarm) ve G17 (çamaşır makinesi rezervasyonu) için kullanım senaryoları aşağıda hazırlanmıştır.

Use Case No	UC-01
Use Case Adı	Arıza Talebi Oluşturma
İlgili Gereksinim	G12, G13, G05
Aktörler	Birincil: Öğrenci · İkincil: Yurt Yöneticisi, Bildirim Sistemi
Açıklama	Öğrenci, odasında oluşan bir arızayı mobil uygulama üzerinden kategori, açıklama ve fotoğraf ekleyerek bildirir.
Ön Koşullar	1. Öğrenci sisteme giriş yapmış olmalıdır. 2. Öğrenci bir odaya kayıtlı olmalıdır.
Tetikleyici	Öğrencinin ana sayfada “Arıza Bildir” butonuna dokunması.
Ana Akış	1. Öğrenci “Arıza Bildir” ekranını açar. 2. Sistem öğrencinin oda bilgisini otomatik olarak doldurur. 3. Öğrenci arıza kategorisini seçer (Elektrik, Su Tesisatı, Mobilya, İnternet, Diğer). 4. Öğrenci arıza açıklamasını yazar. 5. Öğrenci isteğe bağlı olarak fotoğraf ekler. 6. Öğrenci “Gönder” butonuna basar. 7. Sistem bilgileri doğrular ve talebi “Beklemede” durumuyla kaydeder. 8. Sistem öğrenciye talep numarasını gösterir ve yöneticiye bildirim gönderir.
Alternatif Akışlar	3a. Kategori seçilmezse sistem “Lütfen kategori seçiniz” uyarısı verir, akış 3. adıma döner. 4a. Açıklama 10 karakterden kısaysa sistem uyarı verir. 5a. Fotoğraf 5 MB’tan büyükse sistem fotoğrafı reddeder ve uyarı gösterir. 7a. İnternet bağlantısı yoksa sistem talebi kaydetmez, “Bağlantı hatası, tekrar deneyiniz” mesajı gösterir.
Son Koşullar	Talep veritabanına kaydedilmiştir, yönetici listesinde görünür ve öğrenci talebinin durumunu “Taleplerim” ekranından takip edebilir.

Use Case No	UC-02
Use Case Adı	Eşik Değeri Aşımında Alarm Gönderme
İlgili Gereksinim	G14, G15, G20
Aktörler	Birincil: ESP32 Cihazı (sistem aktörü) · İkincil: Yurt Yöneticisi, Oda Sakinleri
Açıklama	Odadaki ESP32 cihazı periyodik olarak ölçüm gönderir; değer belirlenen eşiği aşarsa sistem ilgili kişilere alarm bildirimi gönderir.
Ön Koşullar	1. ESP32 cihazı sisteme kayıtlı ve bir odaya tanımlı olmalıdır. 2. Cihaz Wi-Fi’ye bağlı olmalıdır. 3. Odanın eşik değerleri tanımlanmış olmalıdır.
Tetikleyici	ESP32’nin 30 saniyede bir ölçüm verisini API’ye göndermesi.
Ana Akış	1. ESP32, DHT22 ve MQ-2 sensörlerinden sıcaklık, nem ve gaz değerlerini okur. 2. ESP32 değerleri cihaz kimliğiyle birlikte JSON olarak Web API’ye gönderir. 3. Sistem cihaz kimliğini doğrular ve ölçümü kaydeder. 4. Sistem ölçümü odanın eşik değerleriyle karşılaştırır. 5. Gaz veya sıcaklık değeri eşiği aştığında sistem bir alarm kaydı oluşturur. 6. Sistem yöneticiye ve oda sakinlerine anlık bildirim gönderir. 7. Sistem ESP32’ye “alarm” cevabı döner ve cihaz buzzer/LED ile yerel uyarı verir.
Alternatif Akışlar	3a. Cihaz kimliği geçersizse sistem veriyi reddeder ve hatayı loglar. 4a. Değerler normal aralıktaysa sadece kayıt yapılır, akış biter. 5a. Aynı oda için son 10 dakika içinde açık bir alarm varsa yeni bildirim gönderilmez (tekrarlı bildirimi önlemek için). 2a. Cihaz 5 dakika boyunca veri göndermezse sistem yöneticiye “Cihaz çevrimdışı” uyarısı verir.
Son Koşullar	Ölçüm ve varsa alarm kaydı veritabanında tutulur; yönetici alarmı “İncelendi” olarak işaretleyene kadar alarm açık kalır.

T.C. ADNAN MENDERES ÜNİVERSİTESİ AYDIN MESLEK YÜKSEKOKULU DEĞİŞİMİN GELECEĞİ...	Programı : BİLGİSAYAR PROGRAMCILIĞI Dersin Adı : **Web Programcılığı III** Tarih :	
	Adı – Soyadı : **Berke HARMAN** Okul No : **256001044** İmza	
Öğretim Yılı-Dönemi: 2026-2027 GÜZ	**AraştırmaProjesi-1**	Not:

Use Case No	UC-03
Use Case Adı	Çamaşır Makinesi Rezervasyonu Yapma
İlgili Gereksinim	G17, G06
Aktörler	Birincil: Öğrenci
Açıklama	Öğrenci, bulunduğu binadaki çamaşır makinelerinin doluluk durumunu görür ve uygun bir saat dilimi için rezervasyon yapar.
Ön Koşullar	1. Öğrenci sisteme giriş yapmış olmalıdır. 2. Öğrencinin aktif (henüz kullanılmamış) başka bir rezervasyonu bulunmamalıdır.
Tetikleyici	Öğrencinin "Çamaşırhane" menüsüne girmesi.
Ana Akış	1. Sistem öğrencinin binasındaki makineleri ve durumlarını (boş, dolu, arızalı) listeler. 2. Öğrenci bir makine ve tarih seçer. 3. Sistem seçilen gün için 1 saatlik boş zaman dilimlerini gösterir. 4. Öğrenci uygun bir saat dilimini seçer ve "Rezervasyon Yap" butonuna basar. 5. Sistem seçilen dilimde çakışma olup olmadığını kontrol eder. 6. Sistem rezervasyonu kaydeder ve öğrenciye onay mesajı gösterir. 7. Sistem rezervasyon saatinden 15 dakika önce öğrenciye hatırlatma bildirimi gönderir.
Alternatif Akışlar	1a. Makine "arızalı" durumdaysa seçilemez, gri olarak gösterilir. 2a. Öğrencinin aktif bir rezervasyonu varsa sistem yeni rezervasyona izin vermez ve mevcut rezervasyonu gösterir. 5a. Aynı anda başka bir öğrenci aynı dilimi aldıysa sistem "Bu saat dolu" mesajı verir ve akış 3. adıma döner. 6a. Öğrenci rezervasyonu en geç 30 dakika öncesine kadar iptal edebilir.
Son Koşullar	Rezervasyon kaydedilmiştir; ilgili zaman dilimi diğer öğrencilere "dolu" olarak görünür.

KAYNAKLAR

1. AmazonWeb Services. (t.y.). What's the difference between monolithic and microservices architecture? Erişim adresi: https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/ (Erişim tarihi: 28.10.2026)

2.Cockburn, A. (2001). Writing effective use cases. Boston: Addison-Wesley.

3.de la Torre, C., Wagner, B. ve Rousos, M. (t.y.). .NET microservices: Architecture for containerized .NET applications. Microsoft. Erişim adresi: https://learn.microsoft.com/dotnet/architecture/microservices/ (Erişim tarihi: 28.10.2026)

4.DSDM Consortium. (2014). The DSDM Agile Project Framework handbook (Bölüm 10: MoSCoW Prioritisation). Erişim adresi: https://www.agilebusiness.org/wp-content/uploads/2026/05/DSDM-Agile-Project-Framework-Handbook.pdf (Erişim tarihi: 28.10.2026)

5.Lewis, J. ve Fowler, M. (2014, 25 Mart). Microservices: A definition of this new architectural term. Erişim adresi: https://martinfowler.com/articles/microservices.html (Erişim tarihi: 28.10.2026)

6. Newman, S. (2021). Building microservices: Designing fine-grained systems (2. baskı). Sebastopol, CA: O'Reilly Media.

7.Richardson, C. (2018). Microservices patterns: With examples in Java. Shelter Island, NY: Manning.

8.Richardson, C. (t.y.). Pattern: Monolithic architecture. Erişim adresi: https://microservices.io/patterns/monolithic.html (Erişim tarihi: 28.10.2026)

9.Richardson, C. (t.y.). Pattern: Microservice architecture. Erişim adresi: https://microservices.io/patterns/microservices.html (Erişim tarihi: 28.10.2026)

10.Sommerville, I. (2016). Software engineering (10. baskı). Boston: Pearson. (Bölüm 4: Requirements Engineering)

11.Hanaylı, M. C. Web Programcılığı III ders notları. Adnan Menderes Üniversitesi Aydın Meslek Yüksekokulu.

Süre : Ders saatinde çıktı alınarak teslim edilmelidir.	***Başarılar*** **Öğr.Gör. MehmetCan HANAYLI**